

执行器流

目录

- 背景
- 竞品分析
 - onnxruntime
- 方案设计
 - 方案一：非thread_local方案
 - 折衷设计
 - 备选方案（现有代码O(N)修改，不推荐）
 - 资源管理—ResourceManager
 - Predictor修改
 - 初始化行为
 - Clone行为
 - 获取Stream
 - Tensor修改
 - 异构硬件相关memory处理
 - NaiveExecutor及OperatorBase修改
 - fusion算子
 - 控制流算子
 - 兼容Paddle-TRT
 - 其它后端可扩展性
 - 对其它模块的影响
 - CI拦截
 - IDG模型高优处理
 - 后续计划
 - 开发排期规划
 - 评审收集的问题
 - 方案二：thread_local方案
 - DeviceContextPool适配
 - Predictor适配
 - 四种场景论述

- 单线程单模型
- 单线程多模型
- 多线程单模型
- 多线程多模型
- 方案比选
- 资源最小化
- 接口设计
 - config接口
 - Predictor接口

背景

执行器流指的是：每个执行器拥有自己的stream。在GPU上的预测部署过程中，期望每个执行器跑在单独的流上，这样用户在进行多模型部署的过程中，有机会更充分的利用GPU资源。

实际业务需求：IDG现有部署框架设计上是每个模型跑在单独的CPU线程+GPU流上，IDG框架管理流，将stream传给推理框架。

竞品分析

onnxruntime

根据onnxruntime的接口文档，其在GPU后端上提供以下能力：

1. 执行器流；
2. 用户外部传流；
3. 支持io stream（测试没起到作用，需进一步阅读源码，onnxruntime默认io流和计算流是同一个流（根据issue和相关修复pr描述，不是同一个流似乎存在什么问题，没有默认放开））

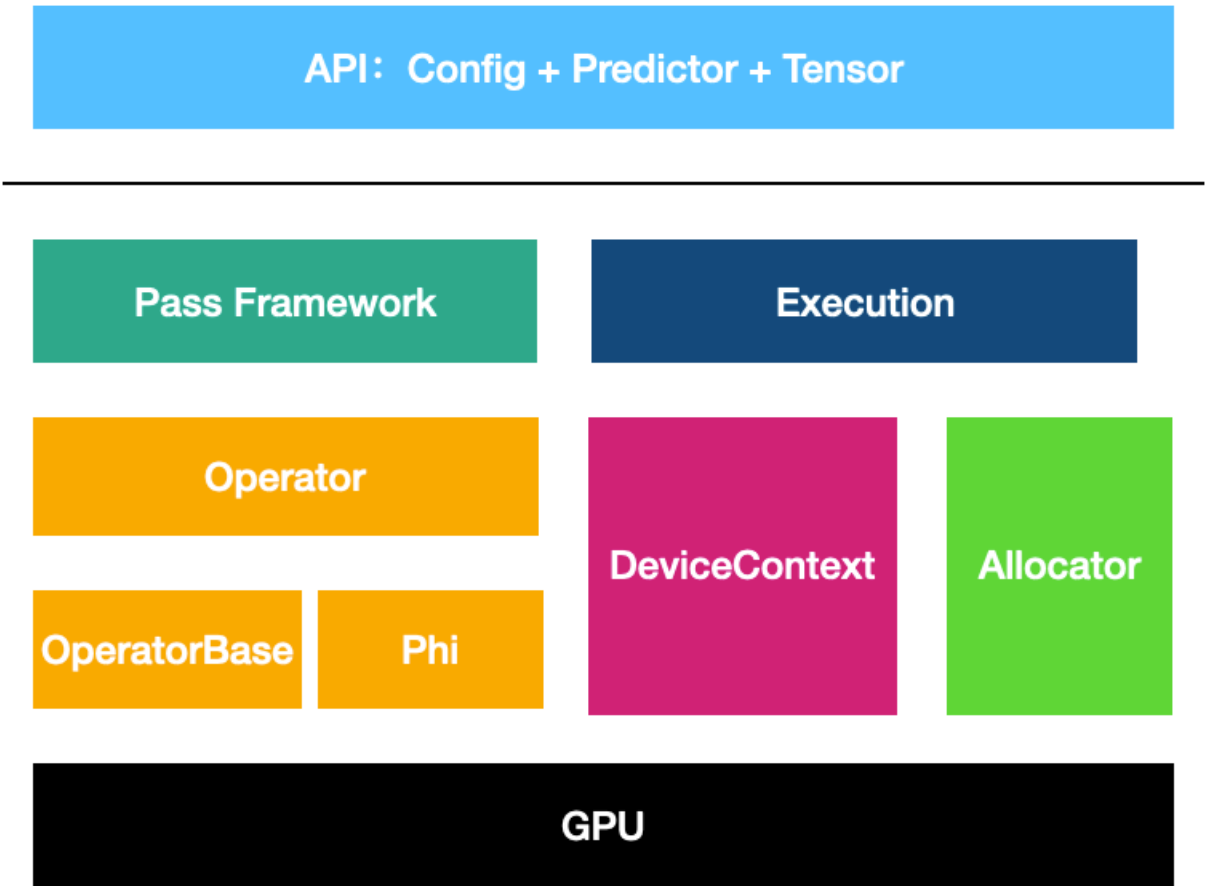
```
/// <summary>
/// Options for the CUDA provider that are passed to SessionOptionsAppendExecutionProvider_CUDA_V2.
/// Please note that this struct is *similar* to OrtCUDAProviderOptions but only to be used internally.
/// Going forward, new cuda provider options are to be supported via this struct and usage of the publicly defined
/// OrtCUDAProviderOptions will be deprecated over time.
/// User can only get the instance of OrtCUDAProviderOptionsV2 via CreateCUDAProviderOptions.
/// </summary>
struct OrtCUDAProviderOptionsV2 {
    int device_id; // cuda device id.
    int has_user_compute_stream; // indicator of user specified CUDA compute stream.
    void* user_compute_stream; // user specified CUDA compute stream.
    int do_copy_in_default_stream; // flag specifying if the default stream is to be used for copying.
    OrtCudnnConvAlgoSearch cudnn_conv_algo_search; // cudnn algo search enum.
    size_t gpu_mem_limit; // BFC Arena memory limit for CUDA.
    onnxruntime::ArenaExtendStrategy arena_extend_strategy; // BFC Arena extension strategy.
    OrtArenaCfg* default_memory_arena_cfg; // (will be overridden by contents of `default_memory_arena_cfg` if it exists)
    int cudnn_conv_use_max_workspace; // BFC Arena config flags.
    int enable_cuda_graph; // flag specifying if maximum workspace can be used in cudnn conv algo search.
    int cudnn_conv1d_pad_to_nc1d; // flag specifying if the CUDA graph is to be captured for the model.
    // flag specifying if pad Conv1D's input [N,C,D] to [N,C,1,D] or [N,C,D,1].
};
```

方案设计

方案一：非thread_local方案

注意：方案的目的是全局DeviceContext修改为Private DeviceContext，目前考虑到phi算子库的完成度等情况，目前只处理GPU后端。

Inference GPU架构如下所示：



我们主要的修改是

- DeviceContext层，从过去的单例模式改成执行器的成员模式；
- Operator算子层；
- Execution调度层；
- API层，提供相关用户接口；
- 相关算子；

折衷设计

现有的Operator包含了两套算子体系：OperatorKernel和phi算子库：

- phi算子库设计之处就有了独立DeviceContext概念，所有的DeviceContext都是phi算子函数式算子的参数，因此能容易的接入；
- OperatorKernel体系，算子从单例访问DeviceContext，较多的operator和functor在使用，很难改动。（[PR-23068](#)是晓伟之前尝试解决执行器流问题提的PR，仅改动了一小部分就涉及

84个文件，一个人很难改动故close掉PR)

与海鹏讨论后，**执行器流仅支持phi算子**，当模型所有算子都运行在phi算子时才启用执行器流，其余情况fallback回退到global DeviceContext运行模式。**(需要尽快迁移fusion算子到phi，控制流算子单独解决)**

实现：

运行之前检测所有GPU算子是否都运行在phi算子上，如果是，则启用Private DeviceContext策略，即启用执行器流，否则回退到Global DeviceContext策略，向前兼容。

备选方案（现有代码O(N)修改，不推荐）

备选方案同晓伟之前改动，代码库中所有用到DeviceContextPool单例的地方均需check和修改，涉及大量文件。

- 优点：
- 兼容现有模型
- 缺点：
- 改动范围较大

资源管理—ResourceManager

Predictor需管理相关资源，构建和析构过程主动申请或释放资源。

提供资源管理类，负责CPU, GPU以及其它后端需要的资源（目前仅支持CPU和GPU）。

Predictor修改

初始化行为

修改相应初始化逻辑，当所有gpu算子均运行在phi上时，启用Private DeviceContexts策略，初始化资源管理类，初始化DeviceContexts。

Clone行为

如果用户设置了外部传流，则每一次Clone都需要指定Stream，否则Predictor的资源管理类负责管理stream。

</> C++ | 收起 ^

```
1 // 使用内部Stream, Clone
2 predictor = CreatePredictor(config);
```

```
3 auto* p = predictor->Clone()
4
5
6 // 外部用户设置Stream后Clone
7 config.SetExecStream(stream1);
8 predictor = CreatePredictor(config);
9 ....
10 auto* p = predictor->Clone(stream)
```

获取Stream

当多模型级联时，级联模型需要运行在同一个stream上以避免不必要的io copy，因此需要有获取Stream的接口。**TODO：CPU XPU 没有stream，接口如何设计？**

</>

C++ | 收起 ^

```
1 // return stream or nullptr(CPU or XPU)
2 void* GetExecStream() const;
3
4 // 未来IO流可能会增加以下接口
5 void* GetInputStream() const;
6 void* GetOutputStream() const;
```

Tensor修改

CopyFromCpu和CopyToCpu接口需获取predictor内部的stream。

异构硬件相关memory处理

当相邻的算子运行在不同的异构硬件上时（一般是指CPU和GPU），框架需显式进行memory::Copy（d2h或h2d的拷贝）此处需保证流的一致性。

现有执行器调度是和训练静态图共用的，训练静态图的异构硬件拷贝是**同步**的方式，不适用于推理场景（cpu->gpu异步，gpu->cpu同步）；此处因为是同步的方式，故无需修改。但为了更优的性能，可以考虑推理单独写执行器。

NaiveExecutor及OperatorBase修改

Paddle Inference现有执行器非常简单，直接按照拓扑序运行即可。

</>

C++ | 收起 ^

```
1 for (auto op : ops) {
2     ...
3     op->Run();
}
```

```
4    ...
5 }
```

现有op的运行直接获取Global DeviceContext，需修改成从参数列表中取Private DeviceContext，因此执行器Executor和算子Operator均需新增Private DeviceContexts参数。

执行器修改Run函数签名：

</>

C++ | 收起 ^

```
1 void Run();
2
3 void Run(const std::map<phi::Place,
  std::shared_future<std::unique_ptr<phi::DeviceContext>>>
  *device_contexts);
```

考虑到Operator runimpl为虚函数，被所有paddle op继承并重写，直接改runimpl函数签名十分困难，因此新增设置Private DeviceContexts函数接口：

</>

C++ | 收起 ^

```
1 void SetDeviceContexts(
2     const std::map<phi::Place,
3     std::shared_future<std::unique_ptr<phi::DeviceContext>>> *
4     device_contexts);
5
6 phi::DeviceContext* GetDeviceContext(const phi::Place& place) const;
```

执行器在运行Op之前设置其使用的DeviceContexts。

fusion算子

fusion类算子有两种处理策略：

1. fusion op迁移phi算子；
2. 在现有的OperatorKernel体系下，针对fusion算子进行O(N)修改。

方式1是期望的解决方法，但需要人力成本高。

方式2也可以解决问题，不过需要维护一个白名单，比较ugly。

控制流算子

phi算子目前没有迁移控制流算子的计划，因此控制流算子必须O(N)处理。

控制流算子会涉及到framework::Executor执行器，因此该执行器也需进行适配。

兼容Paddle-TRT

Paddle-TRT tensorrt_engine 算子处理同控制流算子。

所有plugin算子接口暴露DeviceContext，需O(n)处理

其它后端可扩展性

可直接扩展到其它后端：

- 对于无stream的AI加速硬件如xpu，如果phi算子库xpu迁移完成后，只需对应修改资源管理类和predictor初始化逻辑即可；
- 对于有stream的AI加速硬件如npu，按照gpu的修改逻辑对应修改即可。

对其它模块的影响

目前仅当所有gpu算子均在phi上运行时，才会启用Private DeviceContext，启用执行器流，其它所有情况仍然使用Global DeviceContext，对其它模块均无影响。

CI拦截

todo

IDG模型高优处理

todo

后续计划

1. thread_local模式实现执行器流功能的移除；
2. phi算子迁移完成后，去除折衷设计引入的代码；
3. 异构硬件memory::copy优化；

开发排期规划

1	功能	时间	备注
2	config相关API	0.5天	Done
3	predictor相关	1天	
4	tensor相关	2h	Done
5	执行器相关	2h	Done
6	OperatorKernel相关	2h	Done
7	memory相关	0.5天	Done

8	资源管理类	1.5天	Done
9	所有gpu算子是否均运行在phi算子库	4h	Done
10	CI及单测问题调试	1天	
11	fusion算子处理	需要O(N)解决	O(N)
12	控制流算子解决	控制流算子数量有限（condition_block，while，compare等），且会涉及训练执行器，单独处理。预计1~2天。	
13	trt plugin convert	需要O(N)	O(N)
14	python接口	1~2天	
15	测试及提测	3天	

评审收集的问题

- 1、新执行器如何支持的多stream？依赖phi算子？ todo
- 2、pytorch是否支持执行器流？ todo
- 3、多个执行器Set同一个流，只申请一份资源？通过资源管理实现。
- 4、thread_local方式是否能解DeviceContext单例的问题？ Predictor.Run()内部hold thread_local变量，见方案二。

方案二： thread_local方案

总体思想： predictor.Run()的调用过程中，会多次从global DeviceContextPool中取DeviceContext。通过c++11关键字thread_local，改变DeviceContextPool的行为，保证每次Pool中取到的DeviceContext和Predictor绑定。

DeviceContextPool适配

增加thread_local指针字段，支持外部设置DeviceContext。

</>C++ | 收起 ^

```
1 class DeviceContextPool {
2   public:
3     explicit DeviceContextPool(const std::vector<platform::Place>&
      places);
4
5     static DeviceContextPool& Instance() {
```

```
6     PADDLE_ENFORCE_NOT_NULL(pool,
7                               platform::errors::PreconditionNotMet(
8                                   "Need to Create DeviceContextPool
firstly!"));
9     return *pool;
10 }
11
12 /*! \brief Create should only called by Init function */
13 static DeviceContextPool& Init(const std::vector<platform::Place>&
places) {
14     if (pool == nullptr) {
15         pool = new DeviceContextPool(places);
16     }
17     return *pool;
18 }
19
20 static void SetPool(DeviceContextPool* dev_pool) { pool = dev_pool; }
21
22 /*! \brief Return handle of single device context. */
23 platform::DeviceContext* Get(const platform::Place& place);
24
25 template <typename Place>
26 const typename DefaultDeviceContextType<Place>::TYPE* GetByPlace(
27     const Place& place) {
28     return reinterpret_cast<
29         const typename DefaultDeviceContextType<Place>::TYPE*>
30     (Get(place));
31 }
32
33 size_t size() const;
34
35 const std::map<Place,
std::shared_future<std::unique_ptr<DeviceContext>>>&
36 device_contexts() const;
37
38 static void SetDeviceContexts(
39     const std::map<Place,
40     std::shared_future<std::unique_ptr<DeviceContext>>>*>);
41 private:
42     static DeviceContextPool* pool;
43     std::map<Place, std::shared_future<std::unique_ptr<DeviceContext>>>
44     device_contexts_;
```

```

45 static thread_local const std::map<
46     Place, std::shared_future<std::unique_ptr<DeviceContext>>>*
47     external_device_contexts_; // not owned
48 DISABLE_COPY_AND_ASSIGN(DeviceContextPool);
49 };
50

```

Predictor适配

Run()函数中需要显式设置DeviceContext。

```

bool AnalysisPredictor::ZeroCopyRun() {
    #if defined(PADDLE_WITH_DISTRIBUTE) && defined(PADDLE_WITH_PSCORE)
        if (config_.dist_config().use_dist_model()) {
            VLOG(3) << "ZeroCopyRun will use the fleet executor.";
            inference::Timer timer;
            timer.tic();
            fleet_exe_>Run(config_.dist_config().carrier_id());
            VLOG(3) << "Fleet executor inf runs once use: "
                << std::to_string(timer.toc()) << "ms";

            return true;
        }
    #endif

    + if (private_context_)
    +     paddle::platform::DeviceContextPool::SetDeviceContexts(&device_contexts_);
    +

    paddle::platform::SetNumThreads(config_.cpu_math_library_num_threads());
    #ifdef PADDLE_WITH_MKLDNN
        if (config_.use_mkldnn_) {
            std::vector<std::vector<int>> shape_vector;
            auto names = GetInputNames();
            for (size_t i = 0; i < names.size(); ++i) {
                auto in_tensor = GetInputTensor(names[i]);
                shape_vector.emplace_back(in_tensor->shape());
            }
            MklDnnPreSet(shape_vector);
        }
    #endif
    executor_>Run();
}

```

四种场景论述

predictor拥有自己的DeviceContext， private DeviceContext， Run()函数调用时， 设置DeviceContextPool引用private DeviceContext。

单线程单模型

```
Predictor1.Run() {  
  ...  
  SetDeviceContext(ctx1);  
  ...  
}
```

```
DeviceContextPool::Get() {  
  ...  
  retrun ctx1;  
  ...  
}
```

单线程多模型

```
Predictor1.Run() {  
  ...  
  SetDeviceContext(ctx1);  
  ...  
}
```

```
Predictor2.Run() {  
  ...  
  SetDeviceContext(ctx2);  
  ...  
}
```

```
DeviceContextPool::Get() {  
  ...  
  retrun ctx;  
  ...  
}
```

多线程单模型

thread1

```
Predictor1.Run() {
  ...
  SetDeviceContext(ctx);
  ...
}
```

```
DeviceContextPool::Get() {
  ...
  retrun ctx;
  ...
}
```

thread2

```
Predictor1.Run() {
  ...
  SetDeviceContext(ctx);
  ...
}
```

```
DeviceContextPool::Get() {
  ...
  retrun ctx;
  ...
}
```

多线程多模型

thread1

```
Predictor1.Run() {
  ...
  SetDeviceContext(ctx);
  ...
}
```

```
Predictor2.Run() {
  ...
  SetDeviceContext(ctx);
  ...
}
```

```
DeviceContextPool::Get() {
  ...
  retrun ctx;
  ...
}
```

thread2

```
Predictor1.Run() {
  ...
  SetDeviceContext(ctx);
  ...
}
```

```
Predictor2.Run() {
  ...
  SetDeviceContext(ctx);
  ...
}
```

```
DeviceContextPool::Get() {
  ...
  retrun ctx;
  ...
}
```

方案比选

1		方案一	方案二：thread_local
2	改动量	需要O(n)	少
3	是否易理解	容易理解	较困难
4	可扩展能力	可扩展模型多流	不可扩展模型多流

方案二改动量较小，且能满足业务需求；在phi算子未完全ready的情况下，可按照方案二执行。

资源最小化

方案一和方案二均为Private Context策略，会遇到以下问题，如果多个predictor设置的同一stream，此时会初始化多份gpu资源，这个是不合理的，因此需要资源管理解决该问题。

通过引用计数的方式控制资源的初始化和析构，详见ResourceManager文件。

接口设计

config接口

提供用户外部设置stream的接口。

</>

C++ | 收起 ^

```
1  ///  
2  ///  
3  ///  
4  ///  
5  void SetExecStream(void* stream);  
6  
7  ///  
8  ///  
9  into a  
10 ///  
11 void* GetExecStream() const;  
12  
13 ///  
14 ///  
15 operation must use the external stream, otherwise the framework  
16 manages  
17 the  
18 stream internally.  
19 ///  
20 bool UseExternalStream() const;
```

Predictor接口

获取Stream接口

</>

C++ | 收起 ^

```
1 // return stream or nullptr(CPU or XPU)
2 void* GetExecStream() const;
3
4 // 未来IO流可能会增加以下接口
5 void* GetInputStream() const;
6 void* GetOutputStream() const;
```

Clone接口

</>

C++ | 收起 ^

```
1 void Clone(void* stream=nullptr);
```